PROGRAMACIÓN AVANZADA

Sesión 17

MergeSort

✎ NOTAS DEL PROFESOR

Índice

1 Objetivos de la sesión **2**

2 Prerrequisitos **2**

3 La actividad física — La fila del salón **2**
3.1 Preparación (antes de clase) 2
3.2 Desarrollo 3

4 Secuencia de clase **3**
4.1 Hook histórico (10 min) 3
4.2 El algoritmo (20 min) 4
4.3 Complejidad y límite inferior (15 min) 4
4.4 Comparación y debate (dentro de los 15 min anteriores) 4
4.5 Implementación en el cuaderno (30 min) 5

5 Errores conceptuales frecuentes **5**

6 Soluciones a los ejercicios **6**
6.1 Ejercicio 1 — Traza manual de `[5,3,8,1,4,2,7,6]` 6
6.2 Ejercicio 2 — Contar comparaciones 6
6.3 Ejercicio 3 — Orden descendente 7
6.4 Ejercicio 4 — MergeSort con comparador 7

7 Conexiones con temas posteriores **8**

8 Material de la sesión **8**

9 Rúbrica de ejercicios **9**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar el patrón divide y vencerás con sus propias palabras.
2. Identificar el caso base y el caso recursivo de MergeSort.
3. Implementar `merge` y `mergesort` en C++ con paso por referencia.
4. Justificar por qué la complejidad es $\mathcal{O}(n \log n)$.
5. Argumentar (sin demostración formal) por qué no puede existir un algoritmo de comparaciones más rápido.
6. Comparar MergeSort con QuickSort y elegir el adecuado según el contexto.

90 minutos. Distribución sugerida:
15 min — actividad física con los alumnos (la fila)
10 min — hook histórico + la pregunta del límite inferior
20 min — explicación del algoritmo: árbol de llamadas + merge
15 min — análisis de complejidad + tabla comparativa
30 min — implementación guiada en el cuaderno Jupyter
Los ejercicios quedan como tarea.

Prerrequisitos

- Recursión: caso base, caso recursivo, call stack (sesión 11)
- Apuntadores y arrays como parámetros (sesión 14)
- Paso por referencia; diseño de API con `int*` (sesión 15)
- Bubble Sort con comparador parametrizado (sesión 15–16)

Los alumnos que tengan dificultades con la recursión lo notarán en la traza del árbol de llamadas. Una señal de alerta: si el alumno no puede predecir en qué orden se ejecutan las dos llamadas recursivas, detenerse y repasar el call stack con un ejemplo pequeño (`mergesort(arr, 0, 1)`) antes de continuar.

La actividad física — La fila del salón

- **Preparación (antes de clase)**

Prepara 8 tarjetas con los números **20, 31, 83, 92, 14, 42, 51, 63**. Repártelas al azar entre ocho voluntarios cuando empiece la clase.

• Desarrollo

1. Pide a los 8 voluntarios que se paren en fila en el centro del salón *en el orden en que recibieron sus tarjetas* (desordenados).
2. Pregunta: "¿Cómo los ordenamos?" Acepta ideas. Luego di: "Vamos a usar la misma estrategia que inventó von Neumann en 1945."
3. Divide la fila físicamente por la mitad: los 4 de la izquierda se separan de los 4 de la derecha.
4. Divide cada grupo de 4 en dos grupos de 2. Luego cada grupo de 2 en dos individuos. Ahora hay 8 filas de 1 persona.
5. Pregunta: "¿Está ordenada una fila de una sola persona?" La respuesta es sí — eso es el caso base.
6. Guía la fusión de dos individuos en un par ordenado: el de menor número se coloca a la izquierda. Repite con todos los pares.
7. Fusiona los pares en grupos de 4 ordenados, comparando los frentes de cada fila. Usa la metáfora: "El de frente de cada fila se presentan, y el menor entra primero."
8. Fusiona las dos filas de 4 en la fila final de 8, ordenada.

¿Cuánto tardó? Con 8 personas, 3 rondas de fusión. Con 16, habrían sido 4 rondas. Con 1 024 personas, solo 10 rondas. Haz que los alumnos calculen $\log_2(8) = 3$ en el momento — ahí nace la intuición de $n \log n$.

Señal de que la actividad funcionó: los alumnos deben poder describir solos el algoritmo en palabras antes de ver el código. Pide a uno que lo describa: si lo logra, la actividad cumplió su propósito.

Secuencia de clase

• Hook histórico (10 min)

Después de la actividad, conecta con von Neumann: "Lo que acaban de hacer lo inventó John von Neumann en 1945, no con personas sino con cintas magnéticas. La limitación no era velocidad sino memoria: los datos no cabían en RAM."

Planta la pregunta que motivará la sección de complejidad: "¿Podemos hacer algo mejor que $n \log n$? ¿O hay un límite físico?" No respondas todavía.

"Si tengo n elementos completamente desordenados, ¿cuántas preguntas de sí/no necesito para saber su orden correcto?"
Respuesta esperada: los alumnos dirán n, $n-1$, n^2... Nadie dirá $\log(n!)$. Eso está bien — es lo que vas a demostrar.
Contraejemplo útil: "Si tienes dos cartas boca abajo, ¿cuántas comparaciones necesitas para ordenarlas?" (1). "¿Y con tres?" (hasta 3). "¿Y con n?" — deja

que la discusión lleve a la intuición de que crece con n.

• El algoritmo (20 min)

Dibuja el árbol de llamadas en el pizarrón para el arreglo de 8 elementos. Construye la función `mergesort` en vivo, primero el esqueleto (caso base y las dos llamadas recursivas), luego agrega `merge`.

Para `merge`, traza manualmente la primera fusión (`[20]` y `[31]`) antes de escribir el código. El patrón "compara los dos frentes, toma el menor" debe quedar claro antes de verlo como código.

El error más común en la implementación de `merge` es olvidar el último `for` que copia `temp` de regreso a `arr`. El alumno escribe el merge en `temp` correctamente, pero `arr` no cambia.

Ejercicio preventivo: antes de escribir el código, pregunta: "¿Dónde está el resultado de merge — en `temp` o en `arr`?" Fuerza la respuesta *en* `arr`, y luego pregunta cómo se logra eso.

El segundo error más común es usar `j = mid` en lugar de `j = mid + 1` al inicializar el índice de la mitad derecha. Resultado: el elemento en `arr[mid]` se procesa dos veces.

Pregunta de verificación antes de ejecutar: "¿Qué cubre la mitad izquierda? ¿Y la derecha?" Espera que el alumno diga `[left, mid]` y `[mid+1, right]` con los corchetes correctos.

• Complejidad y límite inferior (15 min)

Con el árbol del pizarrón todavía visible, pregunta cuántos niveles tiene (3 para 8 elementos). Cuántos elementos se procesan en cada nivel (8 en total, repartidos). Multiplica: $3 \times 8 = 24 \approx n \log n$.

Para el límite inferior: plantea el argumento de las permutaciones. "¿Cuántas formas hay de ordenar 3 cartas?" (6 = 3!). "¿Cuántas preguntas de sí/no necesitamos para distinguir 6 casos?" ($\log_2 6 \approx 2.58$, redondeado a 3). Extiende: $n!$ permutaciones, $\log_2(n!)$ preguntas mínimo.

No es necesario demostrar la aproximación de Stirling — basta con que los alumnos vean que $\log(n!)$ y $n \log n$ son del mismo orden para valores concretos ($n = 8$: $\log(8!) = \log(40320) \approx 15.3$; $8 \log 8 = 24$).

• Comparación y debate (dentro de los 15 min anteriores)

Presenta la tabla comparativa brevemente. Lanza la pregunta: "¿MergeSort es el mejor?" y permite que los alumnos argumenten. Guía hacia la conclusión: *depende del contexto*.

"Si MergeSort garantiza $\mathcal{O}(n \log n)$ y QuickSort puede ser $\mathcal{O}(n^2)$, ¿por qué `std::sort` usa QuickSort?"
Respuesta: QuickSort tiene mejor *localidad de referencia* — accede a memoria contigua más seguido, lo que lo hace más amigable con la caché del CPU. En la práctica, ese efecto supera la diferencia teórica de complejidad. `std::stable_sort` sí usa MergeSort porque la estabilidad la requieren.

- **Implementación en el cuaderno (30 min)**

Deja que los alumnos implementen `merge` solos primero (10 min). Si el grupo se atasca, da la pista del índice `k`: "Piensa en `k` como el cursor en `temp` — avanza cada vez que insertas un elemento."

Luego `mergesort` (5 min — es más corta). Finalmente la celda de prueba.

Si el tiempo lo permite, agrega una versión de `mergesort` que imprime el estado del arreglo antes de cada llamada a `merge`. Los alumnos pueden ver cómo el arreglo se va ordenando "de abajo hacia arriba". Esta versión es excelente para despejar confusiones sobre el orden de ejecución.

```
void mergesort_traza(int* arr, int left, int right, int nivel = 0) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    mergesort_traza(arr, left,    mid, nivel + 1);
    mergesort_traza(arr, mid + 1, right, nivel + 1);

    for (int i = 0; i < nivel; i++) cout << "  ";
    cout << "merge [" << left << "," << right << "]: ";
    merge(arr, left, mid, right);
    for (int i = left; i <= right; i++) cout << arr[i] << " ";
    cout << endl;
}
```

Errores conceptuales frecuentes

1. "¿Por qué necesitamos el buffer temporal? ¿No podemos ordenar in-place?"
La fusión in-place es posible pero tiene complejidad $\mathcal{O}(n \log^2 n)$ y es mucho más difícil de implementar. El buffer de $\mathcal{O}(n)$ es el trade-off estándar. Para sistemas con memoria muy restringida (embedded) existe HeapSort que ordena in-place en $\mathcal{O}(n \log n)$.

2. "¿El `vector<int> temp(n)` no es muy costoso de crear en cada llamada?"
En la práctica, los compiladores modernos optimizan allocations pequeños y frecuentes. Sin embargo, es una observación válida: la versión de producción de MergeSort pre-aloca el buffer una sola vez en la función principal y lo pasa como parámetro a todas las llamadas. Este es un buen ejercicio de diseño de API: agregar un parámetro `int* temp` a `merge`.

3. "¿Por qué los índices y no copias del subarreglo?"
Copiar el subarreglo en cada llamada recursiva usaría $\mathcal{O}(n \log n)$ de memoria adicional en lugar de $\mathcal{O}(n)$. Los índices `left` y `right` delimitan el trabajo sin mover datos.

4. Confundir la recursión de MergeSort con la de factorial
En factorial, la llamada recursiva *depende* del resultado de la llamada anterior. En MergeSort, las dos llamadas recursivas son *independientes entre sí* — se podría ejecutarlas en paralelo. Esto hace a MergeSort paralelizable de forma natural (MapReduce, por ejemplo, es MergeSort distribuido).

5. "¿Por qué QuickSort es $\mathcal{O}(n^2)$ en el peor caso?"
Si el pivote elegido es siempre el mínimo o el máximo, la partición deja un lado con $n-1$ elementos y el otro vacío — el árbol de llamadas degenera en una lista lineal. Con 8 elementos ordenados y pivote en la posición 0: $7+6+5+\ldots+1 = 28$ comparaciones en lugar de 24.

Soluciones a los ejercicios

• Ejercicio 1 — Traza manual de `[5,3,8,1,4,2,7,6]`

Nivel	Subarreglos	Acción
0	`[5,3,8,1,4,2,7,6]`	divide
1	`[5,3,8,1]` `[4,2,7,6]`	divide
2	`[5,3]` `[8,1]` `[4,2]` `[7,6]`	divide
3	`[5]` `[3]` `[8]` `[1]` `[4]` `[2]` `[7]` `[6]`	caso base
2	`[3,5]` `[1,8]` `[2,4]` `[6,7]`	merge pares
1	`[1,3,5,8]` `[2,4,6,7]`	merge grupos
0	`[1,2,3,4,5,6,7,8]`	merge final

• Ejercicio 2 — Contar comparaciones

```
int comparaciones = 0;    // variable global

void merge(int* arr, int left, int mid, int right) {
    int n = right - left + 1;
    vector<int> temp(n);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        comparaciones++;    // contamos aqui
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
```

```
        else                    temp[k++] = arr[j++];
    }
    while (i <= mid)   temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int k = 0; k < n; k++) arr[left + k] = temp[k];
}
```

Para 8 elementos: 17 comparaciones en el arreglo `{20,31,83,92,14,42,51,63}` (varía según el orden; el máximo teórico para $n = 8$ es $n \log_2 n - n + 1 = 17$).

> La predicción $n \log n = 24$ es una cota superior, no el valor exacto. El número real de comparaciones de MergeSort es entre $n/2 \cdot \log n$ y $n \log n - n + 1$ dependiendo de los datos. Para el arreglo dado, el resultado será ≤ 17.

• Ejercicio 3 — Orden descendente

Solo cambia la condición en `merge`:

```
if (arr[i] >= arr[j])   // era <=
    temp[k++] = arr[i++];
else
    temp[k++] = arr[j++];
```

Una línea cambiada. Refleja que el **criterio de ordenamiento** está encapsulado en una sola comparación.

• Ejercicio 4 — MergeSort con comparador

```
void merge(int* arr, int left, int mid, int right,
           bool(*cmp)(int, int)) {
    int n = right - left + 1;
    vector<int> temp(n);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (cmp(arr[i], arr[j])) temp[k++] = arr[i++];
        else                     temp[k++] = arr[j++];
    }
    while (i <= mid)   temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int k = 0; k < n; k++) arr[left + k] = temp[k];
}

void mergesort(int* arr, int left, int right,
               bool(*cmp)(int, int)) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    mergesort(arr, left,    mid,   cmp);
    mergesort(arr, mid + 1, right, cmp);
    merge     (arr, left, mid, right, cmp);
```

```
}

// Uso:
// mergesort(arr, 0, 7, [](int a, int b){ return a <= b; });
```

Conexiones con temas posteriores

- **Sesiones 18–20** — Memoria dinámica: el `vector<int> temp(n)` en `merge` es una introducción suave a la alocación dinámica. La versión con `new int[n]` / `delete[] temp` es equivalente.
- **Sesiones 21–25** — Plantillas (`template<typename T>`): MergeSort genérico que ordena cualquier tipo con un comparador.
- **Concurrencia** (tema avanzado): las dos llamadas recursivas de `mergesort` son independientes — se pueden ejecutar en paralelo con `std::thread` o `std::async`. MergeSort paralelo es uno de los ejemplos clásicos de algoritmos paralelos.
- **STL**: `std::merge` (fusiona dos rangos ordenados en un tercero), `std::inplace_merge` y `std::stable_sort` son MergeSort en la biblioteca estándar.

Material de la sesión

- **Cuaderno:** `sesiones/sesion17_mergesort.ipynb`
- **Notas alumno:** `notas_alumno/pdf/sesion17_mergesort.pdf`
- **Material:** 8 tarjetas con números `{20,31,83,92,14,42,51,63}`

Rúbrica de ejercicios

Criterio	Puntos	Evidencia
Ejercicio 1 — Traza manual		
Árbol de divisiones correcto	5	3 niveles, 8 hojas
Fusiones en orden correcto	5	de abajo hacia arriba
Ejercicio 2 — Contar comparaciones		
Variable global o parámetro de conteo	4	contador incrementado
Contador solo en el `while` mixto	3	no en los `while` de cola
Resultado impreso correctamente	3	valor $\leq n \log n$
Ejercicio 3 — Orden descendente		
Una sola línea cambiada	5	solo la condición de `merge`
Resultado correcto	5	arreglo en orden inverso
Ejercicio 4 — Comparador parametrizado		
Firma correcta con `bool(*cmp)(int,int)`	5	ambas funciones actualizadas
Funciona con lambda ascendente	3	resultado correcto
Funciona con lambda descendente	3	resultado correcto
Sin cambios en el cuerpo del algoritmo	4	solo la condición usa `cmp`
Total	**45**	